Name – Meenakshi

Batch ID – 7943

Task 1:

Steps to create a docker file:

1. Use the ubuntu:latest image.
2. Add the File author/maintainer.
3. Install Nginx (apt-get update && apt-get install nginx -y).
4. Use the **Expose** keyword to document that port 80 is to be exposed while running the container.
5. Use CMD or ENTRYPOINT or both to replace the content of the var/www/index.html file with the IP address of the container and run the Nginx service.
6. Build the image with the name "Nginx-ip" and deploy a container

Verify your work

Deploy a container using the **docker run** command and publish port 80 to host port 8080. When you hit the instance IP with port 8080 from your browser, you should be able to view the IP address instead of the Nginx welcome page.

Solution task 1 -:

DockerFile -:

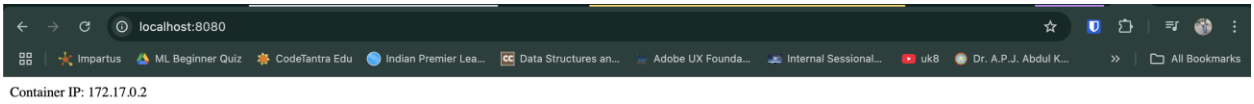


7. Build the image with the name "Nginx-ip" and deploy a container

```
(base) Meenakshis-MacBook-Pro:docker-project meenakshi$ docker build -t nginx-ip .
[+] Building 24.1s (9/9) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from dockerfile                                                                                 0.0s
 => => transferring dockerfile: 259B                                                                                                 0.0s
 => [internal] load metadata for docker.io/library/ubuntu:latest                                                                     3.3s
 => [auth] library/ubuntu:pull token for registry-1.docker.io                                                                        0.0s
 => [internal] load .dockerignore                                                                                                    0.0s
 => => transferring context: 2B                                                                                                      0.0s
 => [internal] load build context                                                                                                    0.0s
 => => transferring context: 87B                                                                                                     0.0s
 => [1/3] FROM docker.io/library/ubuntu:latest@sha256:99c35190e22d294cdace2783ac55effc69d32896daaa265f0bbedbcde4fbe3e5               5.8s
 => => resolve docker.io/library/ubuntu:latest@sha256:99c35190e22d294cdace2783ac55effc69d32896daaa265f0bbedbcde4fbe3e5               0.0s
 => => sha256:99c35190e22d294cdace2783ac55effc69d32896daaa265f0bbedbcde4fbe3e5 6.69kB / 6.69kB                                        0.0s
 => => sha256:4f6ea18f656aa6acbec418aeafc4b99c49ed958a1a47b5cd8cafd22698454500 424B / 424B                                            0.0s
 => => sha256:f825f99d3d8a5d72586ed4d02ba949a7235e37effd735e7e258b5667ec805834 2.31kB / 2.31kB                                        0.0s
 => => sha256:1f630473117188fe348ebdcf0da5e93138275af14007f15baf79967a365e647a 28.89MB / 28.89MB                                     4.9s
 => => extracting sha256:1f630473117188fe348ebdcf0da5e93138275af14007f15baf79967a365e647a                                             0.8s
 => [2/3] ADD maintainer.txt .                                                                                                       0.1s
 => [3/3] RUN apt-get update && apt-get install nginx -y                                                                            14.7s
 => exporting to image                                                                                                               0.1s
 => => exporting layers                                                                                                              0.1s
 => => writing image sha256:7da65e7d45bff32aab6e61866fb5f9c0190e0dfac68f6262b345249d310b620d                                         0.0s
 => => naming to docker.io/library/nginx-ip                                                                                          0.0s
```

Verify your work

Deploy a container using the **docker run** command and publish port 80 to host port 8080. When you hit the instance IP with port 8080 from your browser, you should be able to view the IP address instead of the Nginx welcome page.



Container IP: 172.17.0.2

Task 2 -:

This is a simple Python/Flask application intended to provide a working example of Uber's external API.

https://github.com/uber/Python-Sample-Application

Write a Dockerfile for this application and deploy the container on the host port 7000.

SOLUTION:-

docker-project-task2 > Python-Sample-Application > dockerfile > ...

```
1  FROM python:3.8-slim
2  WORKDIR /app
3  COPY . /app
4  RUN pip install --no-cache-dir -r requirements.txt
5  RUN pip install --no-cache-dir -r requirements-test.txt
6  EXPOSE 7000
7  CMD ["python", "app.py"]
8
```

```
(base) Meenakshis-MacBook-Pro:Python-Sample-Application meenakshi$ docker build -t py-sample-application .
[+] Building 2.1s (11/11) FINISHED
 => [internal] load build definition from dockerfile
 => => transferring dockerfile: 469B
 => [internal] load metadata for docker.io/library/python:3.8-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.8-slim@sha256:1d52838af602b4b5a831beb13a0e4d073280665ea7be7f69ce2382f29c5a613f
 => [internal] load build context
 => => transferring context: 4.53kB
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY . /app
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] RUN pip install --no-cache-dir -r requirements-test.txt
 => exporting to image
 => => exporting layers
 => => writing image sha256:7aba90671174f62916d42e70ed6c2a0108a8958bb1e30aa4ce3e930930a72658
 => => naming to docker.io/library/py-sample-application
```

```
2024-11-05 16:00:05  * Serving Flask app 'app'
2024-11-05 16:00:05  * Debug mode: on
2024-11-05 16:00:05 WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
2024-11-05 16:00:05  * Running on http://127.0.0.1:7000
2024-11-05 16:00:05 Press CTRL+C to quit
2024-11-05 16:00:05  * Restarting with stat
2024-11-05 16:00:05  * Debugger is active!
2024-11-05 16:00:05  * Debugger PIN: 141-486-675
```

Task 3:

Containerize a MERN Stack-based Movie Application using the Source Code URL and Explanation Link.

NOTE:

- The Frontend(client), Backend(server), and MongoDB should be deployed as docker containers on the Ubuntu 20.04 EC2 Instance.
- A Docker custom bridge network should be used for the deployment of all the containers of the Movie Application.
- Use Docker Compose to deploy all the docker containers.
- Map the host machine port 80 with the frontend client running port.
- Use the docker volume to persist the MongoDB data.

Verify the following CRUD operations on the containerized deployment of the Movie application:

1. Create a Movie

2. List the Movie
3. Delete it
4. Update it

SUBMISSION GUIDELINES: Please upload all the Dockerfiles and the YAML files to Google Drive and submit the URL.

SOLUTION -:

1. Log in to your EC2 instance and clone the remote repository on your system using the

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-19-88:~$ git clone https://github.com/samaronybarros/movies-app.git
Cloning into 'movies-app'...
remote: Enumerating objects: 224, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (54/54), done.
remote: Total 224 (delta 79), reused 48 (delta 47), pack-reused 123 (from 1)
Receiving objects: 100% (224/224), 404.39 KiB | 13.94 MiB/s, done.
Resolving deltas: 100% (119/119), done.
ubuntu@ip-172-31-19-88:~$
```

2. Then, move into the **movies-app** directory using the **cd** command.

```
ubuntu@ip-172-31-19-88:~$ cd movies-app/
ubuntu@ip-172-31-19-88:~/movies-app$
```

3. vim ./movies-app/client/src/api/index.js

```
import axios from 'axios'

const api = axios.create({
    baseURL: 'http://52.204.194.190:3000/api',
})

export const insertMovie = payload => api.post(`/movie`, payload)
export const getAllMovies = () => api.get(`/movies`)
export const updateMovieById = (id, payload) => api.put(`/movie/${id}`, payload)
export const deleteMovieById = id => api.delete(`/movie/${id}`)
export const getMovieById = id => api.get(`/movie/${id}`)

const apis = {
    insertMovie,
    getAllMovies,
    updateMovieById,
    deleteMovieById,
    getMovieById,
}

export default apis
```

4. vim ./movies-app/server/db/index.js

```
GNU nano 7.2                                                    index.js *
const { Client } = require('pg');

// Replace these variables with your RDS details
const username = 'postgres';
const password = 'jXnSHJWcc9oeDrer04N6';
const host = 'db-instance-movie.cr12wagcmonw.us-east-1.rds.amazonaws.com';
const port = 5432;  // Default PostgreSQL port
const database = 'movie';

// Create a new client instance
const client = new Client({
    user: username,
    host: host,
    database: database,
    password: password,
    port: port,
});

// Connect to the PostgreSQL RDS instance
client.connect()
    .then(() => console.log('Connected to PostgreSQL on RDS'))
    .catch(e => console.error('Connection error', e.message));

^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^O Where Was
```

5. Configure your security groups to open the required ports 80 and 8000.

Type	Protocol	Port range	Source	Description
Custom TCP	TCP	8000	0.0.0.0/0	–
HTTP	TCP	80	0.0.0.0/0	–
PostgreSQL	TCP	5432	0.0.0.0/0	–
SSH	TCP	22	0.0.0.0/0	–
Custom TCP	TCP	3000	0.0.0.0/0	–

6. **Creating Dockerfiles for server applications**

```
  GNU nano 7.2                                                  Dockerfile
FROM node:10
WORKDIR /opt/app
COPY package*.json ./
COPY . .
RUN CI=true
RUN npm install
EXPOSE 8000
CMD [ "npm", "start" ]
```

7. **Creating Dockerfiles for client applications**

```
  GNU nano 7.2                                                  Dockerfile
FROM node:10
WORKDIR /opt/app
COPY package*.json ./
COPY . .
RUN CI=true
RUN npm install
EXPOSE 3000
CMD [ "node", "index.js" ]
                                                        [ Read 10 lines ]
```

8. **Creating docker-compose.yml files for web applications**

```
GNU nano 7.2                                                    docker-c
services:
  client:
    build: ./client
    container_name: client
    ports:
      - "8000:8000"
    networks:
      - fe-tier

  server:
    build: ./server
    container_name: server
    depends_on:
      postgres:
        condition: service_healthy
    ports:
      - "3000:3000"
    networks:
      - fe-tier
      - be-tier

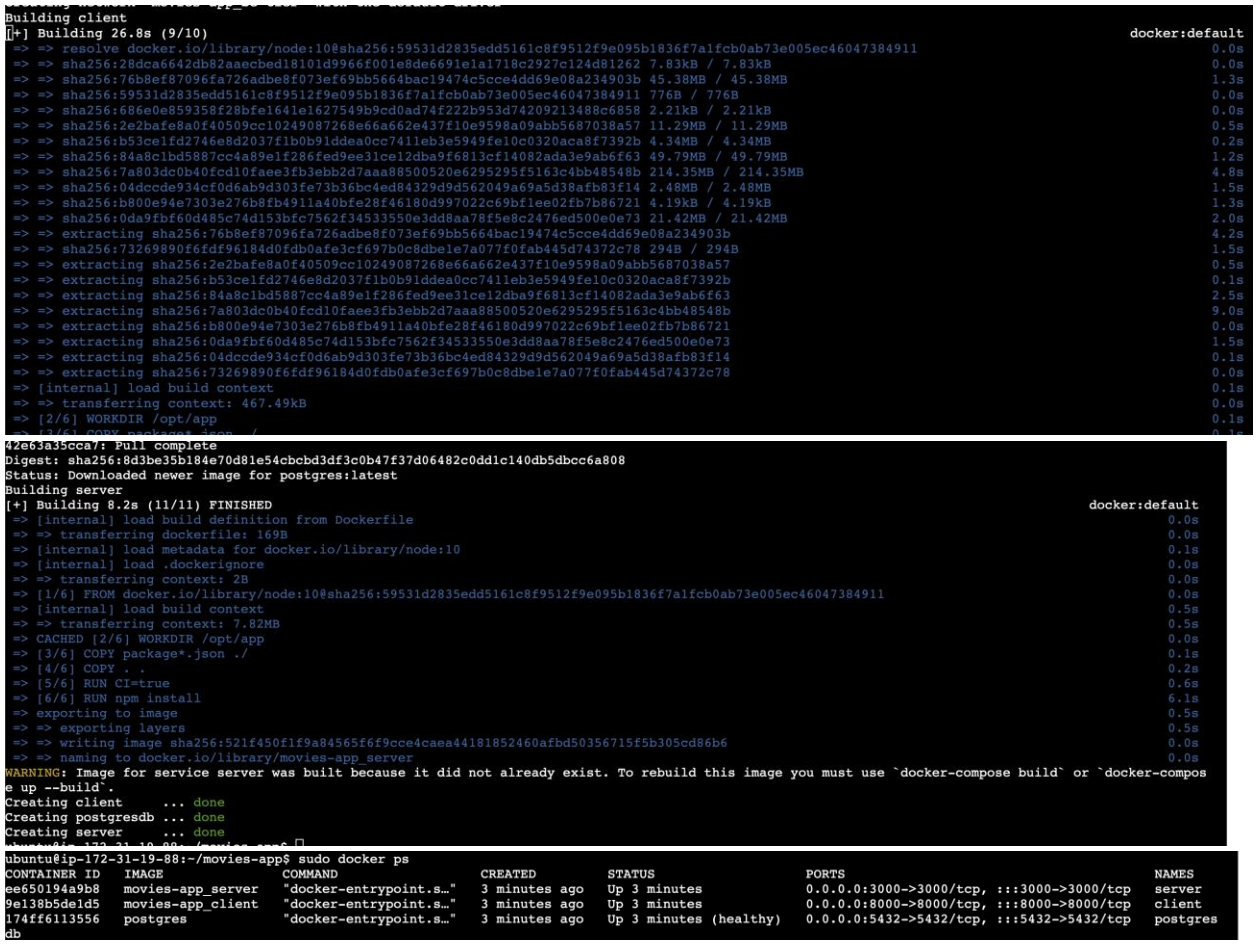
  postgres:
    image: postgres
```

```
      - be-tier

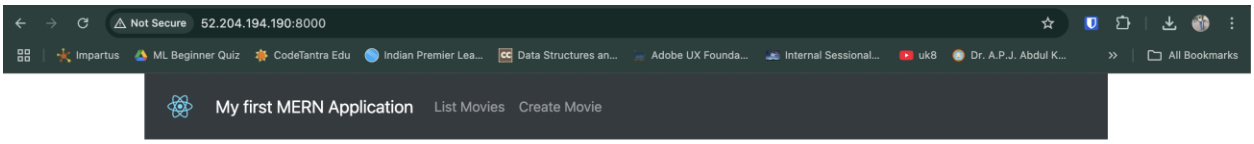
  postgres:
    image: postgres
    container_name: postgresdb
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: jXnSHJWcc9oeDrer04N6
      POSTGRES_DB: movie
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U yourusername"]
      interval: 10s
      timeout: 5s
      retries: 5
    ports:
      - "5432:5432"
    networks:
      - be-tier

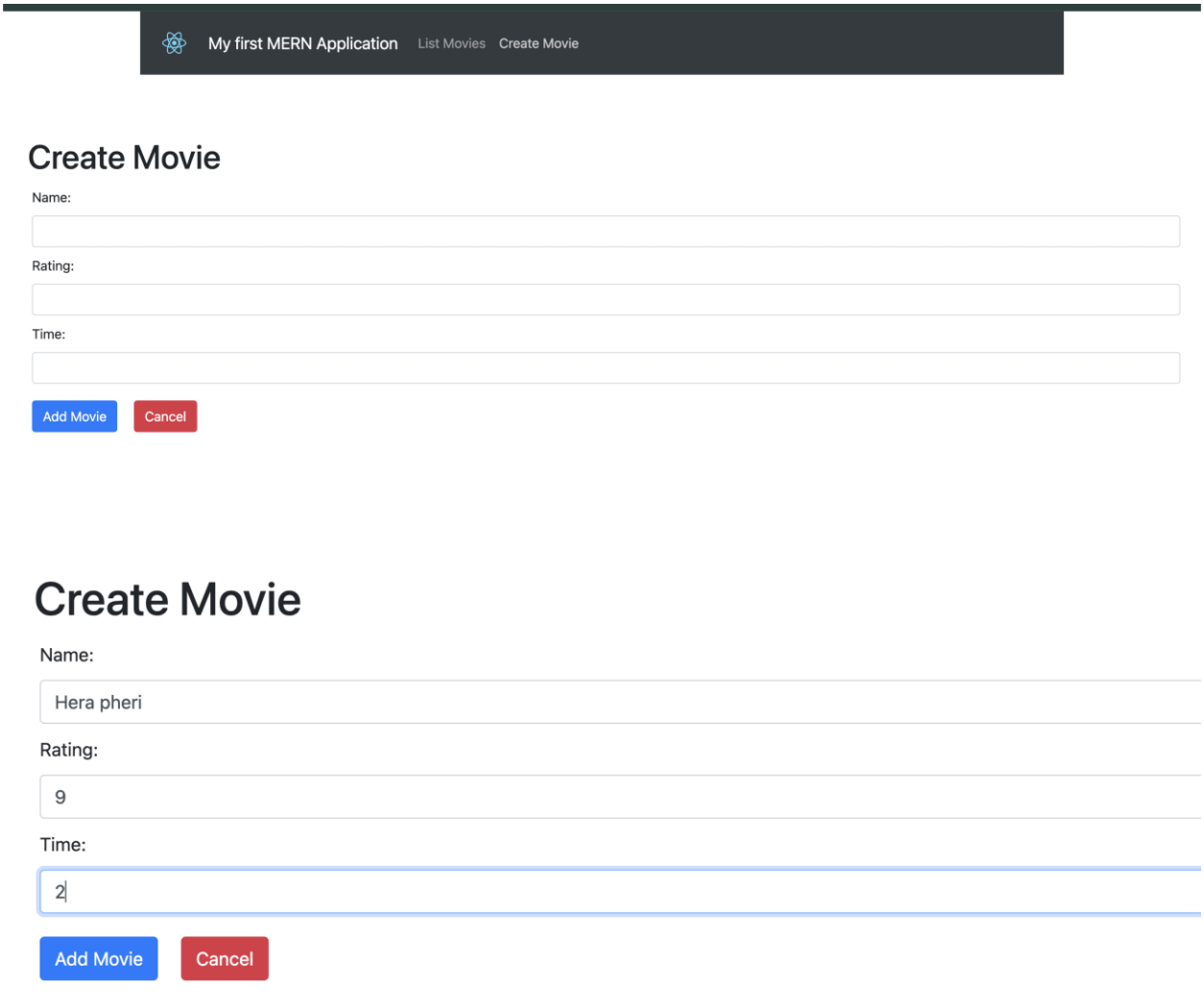
networks:
  fe-tier:
  be-tier:
```

9. Next, **sudo docker-compose up -d** to start the containers.



1. Then, go to your browser and type the URL **http://<ip_address>:8000/**. You can see your app up and running.


Not Secure
52.204.194.190:8000
Impartus
ML Beginner Quiz
CodeTantra Edu
Indian Premier Lea...
Data Structures an...
Adobe UX Founda...
Internal Sessional...
uk8
Dr. A.P.J. Abdul K...
All Bookmarks
My first MERN Application
List Movies
Create Movie


My first MERN Application
List Movies
Create Movie
Create Movie
Name:
Rating:
Time:
Add Movie
Cancel

Create Movie
Name:
Hera pheri
Rating:
9
Time:
2
Add Movie
Cancel